

LifeLink: An AI-Based Blood Donor Management System

Cardiff Metropolitan University

Cardiff School of Technology

BSc (Hons) Software Engineering

LifeLink: An AI-Based Blood Donor Management System with Donor Eligibility Prediction and Blood Matching

Submitted in [TODO: Month Year]

By

Tharsika Kailayapathy Student ID – JF/BSCSD/18/43

This dissertation is submitted in partial fulfilment of the requirements for the degree of Bachelor of Science (Honours) in Software Engineering (BSc SE)

Supervisor: [TODO: Supervisor's Name]

Acknowledgement

I would like to express my sincere gratitude to my supervisor, [TODO: Supervisor's Name], for the guidance, feedback, and patience provided throughout the development of this project. Their direction was invaluable in shaping both the technical implementation and the critical evaluation presented in this report.

I am also grateful to the academic staff of the BSc (Hons) Software Engineering programme at the International College of Business and Technology, delivered in partnership with Cardiff Metropolitan University, for the knowledge and skills that made this work possible.

Finally, I would like to thank my family and friends for their continued encouragement, and to acknowledge the wider motivation behind this project: the donors, hospital staff, and blood transfusion services in Sri Lanka whose day-to-day challenges this system was designed to address.

Abstract

Blood donor management in Sri Lanka remains largely dependent on manual coordination and fragmented record-keeping, which slows the process of matching eligible donors to urgent hospital blood requests and leaves demand planning reactive rather than proactive. This project presents LifeLink, a full-stack, AI-assisted blood donor management system built as a Laravel web application backed by an independent Python machine learning service.

LifeLink allows donors to register and maintain a health profile, hospitals to submit blood requests and receive a ranked list of compatible donors, and administrators to oversee donors, hospitals, and system-wide analytics. Three machine learning capabilities support these workflows: a donor

eligibility classifier trained on donor health and demographic data, a donor-hospital matching mechanism that combines ABO/Rh blood compatibility, geographic proximity, and each donor's predicted reliability, and a short-term blood demand forecasting model. Each AI component was evaluated on held-out test data using standard classification and regression metrics, and the evaluation process itself surfaced and corrected genuine methodological issues in the initial implementation, including a mislabelled classification target and a data-cleaning step that had silently discarded the majority of the available training data.

The current implementation covers donor, hospital, and administrator functionality, is supported by an automated test suite, and has been verified through live end-to-end testing rather than through automated tests alone. This report documents the problem context, the design and development process, the implementation, and a critical evaluation of the system against its original objectives, including an honest account of what has and has not yet been achieved.

Keywords: Blood Donor Management, Artificial Intelligence, Machine Learning, Donor Eligibility Prediction, Donor-Patient Matching, Blood Demand Forecasting, Laravel, Healthcare Logistics, Sri Lanka

Table of Contents

Page numbers to be finalised once the report is complete and paginated for submission.

- Acknowledgement
- Abstract
- **1.0 Introduction**
 - 1.1 Background Studies
 - 1.2 Problem Statement
 - 1.3 Objectives
 - 1.4 Proposed Solution
- **2.0 Literature Review**
 - 2.1 Overview of Blood Donor Management Systems
 - 2.2 Artificial Intelligence in Healthcare
 - 2.3 Machine Learning for Donor Eligibility Prediction
 - 2.4 Blood Demand Forecasting
 - 2.5 Donor Matching and Recommendation Techniques
 - 2.6 Hospital Blood Inventory Management
 - 2.7 Comparison with the LifeLink Approach
 - 2.8 Research Gap and Motivation
 - 2.9 Summary
- **3.0 Planning**
 - 3.1 Feasibility Report
 - 3.2 Risk Assessment
 - 3.3 SWOT Analysis
 - 3.4 PESTEL Analysis
 - 3.5 Life Cycle Model

- 3.6 Time Planning
 - **4.0 Requirement Gathering and Analysis**
 - 4.1 Requirement Analysis Approach
 - 4.2 Requirement Gathering Techniques
 - 4.3 Functional and Non-Functional Requirements
 - **5.0 System Design**
 - 5.1 Architecture Diagram
 - 5.2 ER Diagram
 - 5.3 UML Diagrams
 - 5.4 Wireframe Diagrams
 - **6.0 Implementation**
 - 6.1 Technology Stack
 - 6.2 Design Patterns
 - 6.3 Implementation of the Program
 - **7.0 Testing and Validation**
 - 7.1 Test Plan
 - 7.2 Test Cases
 - 7.3 Automated Test Suite Summary
 - 7.4 AI Model Validation
 - **8.0 Conclusion**
 - 8.1 Conclusion
 - 8.2 Limitations
 - 8.3 Future Recommendations
 - 8.4 Lessons Learned
 - Gantt Chart (*not yet written*)
 - References (*not yet written — will consolidate every citation used across all chapters*)
 - Appendix 1 — Requirements Evidence (*not yet written*)
 - Appendix 2 — Supervision Meeting Logs (*not yet written; [TODO: attach signed logs as they occur]*)
-

List of Figures

Figure 1: System Architecture Diagram (Section 5.1) Figure 2: Entity Relationship Diagram (Section 5.2) Figure 3: Use Case Diagram (Section 5.3) Figure 4: Sequence Diagram — Donor Registration and AI Eligibility Check (Section 5.3) Figure 5: Sequence Diagram — Hospital Blood Request and AI-Assisted Matching (Section 5.3) Figure 6: Wireframe — Donor Dashboard (Section 5.4) Figure 7: Wireframe — Hospital Matched Donors Page (Section 5.4) Figure 8: Wireframe — Admin Dashboard (Section 5.4)

Consistent with the reference report's own convention, no figures appear before Chapter 5. Further figures (implementation screenshots) will be added from Chapter 6 onward as real screenshots become available.

List of Tables

Table 1: Comparison of prior work with the LifeLink approach (Section 2.7) Table 2: Risk Assessment (Section 3.2) Table 3: SWOT Analysis (Section 3.3) Table 4: Project Time Plan (Section 3.6) Table 5: Functional Requirements (Section 4.3.1) Table 6: Non-Functional Requirements (Section 4.3.2) Table 7: Technology Stack (Section 6.1) Table 8: Test Plan (Section 7.1) Table 9: Test Cases (Section 7.2)

Further tables will be added from Chapter 8 onward if required.

1.0 Introduction

Blood transfusion services depend on a continuous, reliable supply of voluntary donors, yet the process of connecting an available donor to a patient in need remains, in many healthcare systems, a manual and time-sensitive undertaking. In Sri Lanka, this challenge is compounded by rising demand from surgical procedures, road traffic accidents, and chronic conditions such as thalassaemia, alongside donor coordination methods that have not kept pace with that demand. LifeLink is an AI-assisted blood donor management system developed to address this gap. The system uses machine learning to predict donor eligibility, to rank donors against a specific hospital blood request by compatibility and reliability, and to forecast short-term blood demand, all delivered through a three-role web application for donors, hospitals, and administrators. This report documents the problem this project addresses, the design and development of the system, and a critical evaluation of what has been built against what was originally proposed.

1.1 Background Studies

The National Blood Transfusion Service (NBTS) of Sri Lanka coordinates blood supply nationally, but day-to-day donor identification and communication still rely heavily on manual and informal channels (National Blood Transfusion Service Sri Lanka, 2022). When a hospital faces an urgent requirement, particularly for a less common blood group, staff frequently resort to phone calls or social media appeals to locate a compatible donor, an approach that is slow and difficult to verify. The World Health Organization has repeatedly identified reliable donor recruitment and retention as a foundational requirement for a safe and sufficient blood supply (World Health Organization, 2021), and the literature on donor behaviour suggests that retention of existing donors is generally more effective, and less costly, than recruiting new ones (Gillespie and Hillyer, 2002) — a pattern that is difficult to act on without infrastructure to track individual donor eligibility and history over time.

Outside of blood services specifically, machine learning has already been applied to related healthcare logistics problems. Hosseini-Motlagh, Cheraghi and Ghatreh-Samani (2020) discuss supply chain frameworks that incorporate planning criteria beyond simple stock counts, while Jain, Singh and Sharma (2019) demonstrate that demand forecasting and donor management can be approached directly as machine learning problems rather than purely operational ones. LifeLink builds on this general direction — applying classification models to donor eligibility and matching, and a forecasting model to demand — but does so for the specific, under-served context of Sri Lankan blood donor coordination, where no equivalent integrated tool currently exists.

1.2 Problem Statement

The current approach to blood donor management in Sri Lanka exhibits several specific limitations:

1. Donor records are typically kept in handwritten registers or disconnected spreadsheets, with no shared digital system allowing donors to be searched in real time by blood group, location, or current eligibility.
2. When a hospital has an urgent blood requirement, locating a compatible donor depends on informal, manual outreach — phone calls or social media — which is slow and unreliable, particularly for rarer blood groups such as AB- or O-.
3. Existing systems provide no forward-looking view of donor availability or blood demand, so stock management remains reactive. Given that donated blood products have a limited usable shelf life, this contributes to both wastage and shortage.
4. Without automated eligibility tracking or personalised engagement, donors who give once are frequently not followed up with, contributing to low rates of repeat donation.

Taken together, these limitations point to a need for a digital platform that can track donor eligibility continuously, rank and surface compatible donors quickly when a request is raised, and provide administrators with a forward-looking view of demand — the gap LifeLink is intended to close.

1.3 Objectives

This project set out to achieve the following objectives:

1. **Donor eligibility prediction** — to design, train, and honestly evaluate a machine learning classifier that predicts whether a donor is currently eligible to donate, based on health and demographic attributes such as age, weight, hemoglobin level, and donation history.
2. **Donor-hospital matching** — to implement a mechanism that, for a specific hospital blood request, ranks compatible donors using blood group and Rh compatibility, geographic proximity, urgency, and each donor's predicted eligibility and reliability.
3. **Blood demand forecasting** — to build a forecasting capability that estimates near-term blood demand per blood group from historical hospital request data, to support proactive rather than reactive planning.
4. **A three-role web portal** — to deliver donor, hospital, and administrator interfaces covering registration, profile management, blood request submission and tracking, and system-wide oversight.
5. **Rigorous, honest evaluation** — to evaluate every machine learning component against held-out data using standard metrics (accuracy, precision, recall, F1-score, and MAPE where applicable), and to report the real performance and limitations of each model rather than an assumed or aspirational one.

1.4 Proposed Solution

LifeLink is implemented as a Laravel web application backed by an independent Python microservice that hosts the machine learning models, the two communicating over a REST API. This

separation allows the donor eligibility, matching, and forecasting logic to be developed, retrained, and evaluated independently of the web application itself.

Donor eligibility is predicted from six features — age, weight, hemoglobin level, total prior donations, blood group, and gender — using a classifier trained on a donor dataset and evaluated on a held-out test split. Donor–hospital matching does not rely on a single machine-learning classifier in isolation; instead, it combines a deterministic ABO/Rh compatibility check, a district-based proximity signal, and the donor’s own eligibility and reliability predictions into a single, explainable match score that is recalculated for every hospital request, rather than reusing a single static per-donor figure. Blood demand forecasting uses each blood group’s recent request history together with the number of currently eligible donors to produce a short-term estimate of upcoming demand.

The web application itself is organised around three role-specific portals — Donor, Hospital, and Administrator — each with its own authentication and access controls, so that a donor cannot access hospital functionality and vice versa. The system is developed and tested incrementally, with an automated test suite alongside manual, live verification of each feature against the running application, and is documented further in the Implementation and Testing chapters of this report.

2.0 Literature Review

This chapter reviews existing work relevant to LifeLink across six areas: blood donor management systems generally, the wider application of artificial intelligence in healthcare, machine learning approaches to donor eligibility prediction specifically, blood demand forecasting, donor matching and recommendation techniques, and hospital blood inventory management. Each section is used to identify what has already been established, and where LifeLink’s approach agrees with, departs from, or extends it. The chapter closes by comparing prior work directly against LifeLink’s implementation and identifying the specific gap this project addresses.

2.1 Overview of Blood Donor Management Systems

A substantial body of prior work has focused on digitising the basic connective function of blood donation — helping a donor and a blood bank or recipient find one another. Chaurasia et al. (2026) describe a web-based platform, coincidentally also named LifeLink, that centralises donors, recipients, and blood banks so that blood availability can be searched by location and blood group and emergency requests can be placed directly. This is a useful point of comparison precisely because it is representative of a common pattern in this literature: most published blood donation platforms prioritise directory-style functionality — registration, search, and request logging — over predictive intelligence. Similar systems built for mobile and web platforms typically add features such as GPS-based donor location and SMS notification, but treat eligibility as something the donor self-reports rather than something the system predicts, and treat donor selection as a search-and-filter operation rather than a ranked recommendation.

This pattern is consistent with the situation described in the LifeLink proposal for the Sri Lankan context specifically (see Chapter 1): digital tools that reduce friction in locating a donor already exist in various forms internationally, but they do not, by themselves, address predictive eligibility, compatibility-aware ranking, or forward-looking demand planning. LifeLink is positioned to extend this connective baseline with the three AI capabilities described in Section 1.4.

2.2 Artificial Intelligence in Healthcare

Artificial intelligence has been adopted unevenly but increasingly widely across healthcare. Ali et al. (2023), in a systematic review synthesising 180 studies, organise AI’s healthcare role into four dimensions — benefits, challenges, methodologies, and functionalities — and find that AI-enabled functionality spans clinical decision-making, diagnosis, treatment planning, patient monitoring, and administrative processes, with consistent evidence of AI improving accuracy and efficiency relative to manual processes in the domains studied. Their review also documents recurring challenges: data quality and availability, integration into existing clinical workflows, and the need for models whose outputs can be explained and trusted by the people using them, not only measured statistically.

Blood donor management sits at the intersection of clinical and administrative healthcare AI — donor eligibility prediction is closer to the clinical/diagnostic use of AI that Ali et al. (2023) describe, while demand forecasting and matching are closer to the administrative and logistics functions. However, relatively little of the AI-in-healthcare literature specifically addresses blood donation as a use case, which is itself part of the gap this project responds to (see Section 2.8).

2.3 Machine Learning for Donor Eligibility Prediction

Several studies have applied machine learning specifically to blood donor prediction problems, although most frame the prediction target as donor *behaviour* — whether a donor will respond, return, or continue donating — rather than *medical* eligibility to donate at a given point in time. Kauten et al. (2021) used a Random Forest classifier on regional blood centre data to predict donor retention, reporting a Matthews correlation coefficient of 0.851, and found that recency, frequency, and volume of past donations were the strongest predictors. Wu et al. (2022) took a similar behavioural framing during COVID-19 donor recruitment campaigns in China, comparing seven machine learning models across 95,476 SMS-based recruitment attempts; XGBoost and Support Vector Machine models performed best (AUC 0.809, F1-score 0.843 for XGBoost), with donation interval, age, and donation frequency again emerging as the strongest predictors.

This is a useful point of contrast for LifeLink. Both studies above predict whether a donor *will* donate again — a behavioural, response-oriented target — using donation-history features almost exclusively. LifeLink’s eligibility classifier instead predicts medical eligibility at the point of registration or profile update, from age, weight, hemoglobin level, blood group, gender, and donation history, against a rule derived from standard donation criteria (minimum age, weight, and hemoglobin thresholds). This is a related but distinct classification target from either of the studies above, and the distinction matters: a system could correctly predict that a donor is *likely to respond* to outreach while being medically ineligible to donate, or vice versa. LifeLink treats these as two separate predictions for this reason — an eligibility classifier and a separate response/

reliability model — rather than conflating them into a single score, which is a design decision this literature does not directly address but which the distinction between the two studies above helps to justify.

2.4 Blood Demand Forecasting

Forecasting blood demand has been studied as a supervised learning problem more directly than eligibility prediction has. Jain, Singh and Sharma (2019) frame blood demand forecasting and donor management jointly as machine learning tasks rather than purely operational planning exercises. Ben Elmir, Hemmak and Senouci (2023) go further, proposing a machine-learning-based platform that forecasts blood supply chain demand to reduce the uncertainty that otherwise causes both wastage and shortage, using historical collection and demand data as its primary input.

A recurring feature of this literature is that the forecasting models are typically trained on multi-year historical datasets enriched with contextual variables — population figures, hospital admission counts, seasonal or weather effects — which is feasible for the large, established blood services these studies are conducted within. LifeLink’s blood demand forecasting was originally designed with a similarly rich feature set in mind, but during development this was found to be infeasible: no equivalent contextual data (population statistics, admissions data) exists anywhere in the LifeLink application, and fabricating placeholder values for a real forecasting model would have produced confident-looking but meaningless predictions. LifeLink’s forecasting was therefore deliberately re-scoped to use only the features the system can genuinely observe — historical weekly request counts per blood group and the number of currently eligible donors — which is a smaller feature set than the literature typically uses, and is discussed critically, including its limitations, in Chapter 7.

2.5 Donor Matching and Recommendation Techniques

Framing donor selection as a ranking or recommendation problem, rather than a simple filtered search, appears less often in the blood donation literature than the eligibility and forecasting problems above, but is not absent. Sivaramakrishnan et al. (2018) propose a blood and organ donation recommendation system for hospital management that calculates donor-to-recipient distance using the Haversine formula and ranks candidate blood banks using the TOPSIS multi-criteria decision-making method, combining proximity with other suitability criteria rather than returning a single nearest match.

LifeLink’s donor–hospital matching engine shares the same underlying idea — that a hospital blood request should return a *ranked* set of candidates rather than an unordered list — but differs in two respects. First, geographic proximity in LifeLink is approximated at district level rather than by geographic coordinates and Haversine distance, since no donor or hospital location data more precise than a district name exists in the system; this is a narrower proximity signal than Sivaramakrishnan et al. (2018) use, and is an explicit, acknowledged limitation rather than an oversight. Second, LifeLink’s ranking combines a deterministic ABO/Rh blood compatibility check with each donor’s own machine-learning-derived eligibility and response predictions, rather than applying a single external multi-criteria ranking method such as TOPSIS to a homogeneous set of already-

compatible candidates. This design choice, and the reasoning behind it, is discussed further in Section 2.7 and in the Implementation chapter.

2.6 Hospital Blood Inventory Management

A separate and long-established strand of research addresses blood inventory management at the level of the blood bank or hospital itself, rather than donor-facing coordination. Meneses, Marques and Barbosa-Póvoa (2023) study ordering policies for hospital blood banks under supply and demand uncertainty, addressing the core trade-off in this literature: holding too little stock risks shortage, while holding too much risks wastage, given that blood products have a limited usable shelf life. Hosseini-Motlagh, Cheraghi and Ghatreh-Samani (2020) similarly examine blood supply chain management, incorporating social and environmental planning criteria alongside the operational ones.

This body of work operates at a different point in the blood supply chain than LifeLink does. LifeLink does not manage physical stock levels, ordering decisions, or wastage directly; it manages the donor-facing side of the chain — eligibility, matching, and demand visibility — which is intended to reduce the *uncertainty* that inventory-management research treats as an input, rather than to replace inventory management itself. This is a genuine scope boundary rather than a gap, and is stated explicitly here so that the two are not conflated.

2.7 Comparison with the LifeLink Approach

Table 1 below summarises how the literature reviewed above compares with LifeLink’s actual implementation.

Table 1: Comparison of prior work with the LifeLink approach

Area	Representative prior work	Typical prior approach	LifeLink’s approach
Donor platforms	Chaurasia et al. (2026)	Directory-style search and manual request logging	Adds predictive eligibility and ranked matching on top of the same core donor/hospital/request model
Eligibility/behaviour prediction	Kauten et al. (2021); Wu et al. (2022)	Predicts behavioural willingness/retention from donation history	Predicts medical eligibility from health and demographic data, kept as a distinct model from response/reliability prediction
Demand forecasting	Jain, Singh and Sharma (2019); Ben Elmir, Hemmak and Senouci (2023)	Rich, multi-year, contextually-enriched datasets (population, admissions)	Deliberately re-scoped to only the features genuinely available (request history, eligible donor counts), acknowledged as a limitation
Donor matching	Sivaramakrishnan et al. (2018)	Coordinate-based distance and external multi-criteria ranking (e.g. TOPSIS)	District-level proximity combined with the system’s own compatibility rules and eligibility/response predictions
Inventory management	Meneses, Marques and Barbosa-Póvoa (2023); Hosseini-Motlagh, Cheraghi and Ghatreh-Samani (2020)	Physical stock ordering policy under uncertainty	Out of scope; LifeLink addresses the donor-facing uncertainty this literature treats as a given input

The overall pattern is that no single prior system reviewed here combines eligibility prediction, compatibility-aware ranked matching, and demand forecasting into one evaluated, end-to-end platform – each has typically been studied, and built, in isolation. LifeLink’s contribution is not a novel algorithm within any one of these areas individually, but the integration of all three into a single system for a specific, currently under-served context.

2.8 Research Gap and Motivation

Drawing the sections above together, three gaps motivate this project. First, existing blood donation platforms (Section 2.1) provide connectivity but little predictive intelligence, while the machine learning studies that do exist for donor prediction (Section 2.3) and demand forecasting (Section 2.4) are largely standalone research contributions rather than components embedded in a deployed, donor-facing system. Second, donor matching framed explicitly as a ranking or recommendation problem (Section 2.5) is comparatively rare in the blood donation literature specifically, most of the relevant technique having been developed and validated in the adjacent context of organ allocation rather than blood. Third, and specific to this project’s context, none of the literature reviewed is set in Sri Lanka; the operational description of manual, phone- and social-media-based donor coordination in Section 1.2 is corroborated by the National Blood Transfusion Service’s own reporting (National Blood Transfusion Service Sri Lanka, 2022) rather than by an existing academic study of the Sri Lankan case specifically.

LifeLink is motivated by, and positioned to address, this combination of gaps: an integrated system, rather than isolated components, applied to a specific national context in which no equivalent tool currently exists.

2.9 Summary

This chapter has reviewed blood donor management platforms, artificial intelligence in healthcare generally, machine learning approaches to donor eligibility and behaviour prediction, blood demand forecasting, donor matching and recommendation techniques, and hospital blood inventory management, comparing each against LifeLink’s own design and implementation. The review shows that LifeLink’s individual techniques are each grounded in an existing research direction, while its combination of eligibility prediction, compatibility-aware matching, and demand forecasting into a single evaluated system, for the Sri Lankan context, is not directly addressed by the literature reviewed. The next chapter sets out how this project was planned, including its feasibility, risk assessment, and development timeline.

3.0 Planning

This chapter sets out how the LifeLink project was planned before and during development: its feasibility across technical, economic, time, and operational dimensions; the risks anticipated and, in several cases, actually encountered; a SWOT and PESTEL analysis of the project’s position; the development methodology followed; and the time plan against which progress was tracked.

3.1 Feasibility Report

Technical feasibility. LifeLink was built entirely from open-source, well-documented technologies — Laravel for the web application, Python with scikit-learn for the machine learning components, and FastAPI to expose them as a REST service. Each of these is mature and widely used individually; the technical risk in this project was less about any single technology and more about integrating a PHP web application with an independently-running Python service reliably, including when that service is slow or unavailable. This risk was real rather than theoretical:

during development, the admin dashboard was found to take approximately 33 seconds to load whenever the AI service was unreachable, because it attempted the same failed network call repeatedly across multiple blood groups. This was resolved by short-circuiting further attempts within a single request once one call had failed, which is discussed further in the Implementation chapter. Technical feasibility, in other words, has been demonstrated in practice, not only argued in advance.

Economic feasibility. The entire technology stack is free and open-source, and the system can be hosted at no cost during development using local or free-tier infrastructure. No proprietary licences, paid APIs, or paid datasets were required at any point.

Time feasibility. The project was planned around the phased schedule set out in Section 3.6, allocating time for requirement gathering, design, AI development, front-end development, integration and testing, and report writing across an academic year. Development in practice proceeded iteratively rather than in strict sequential phases, discussed further in Section 3.5.

Operational feasibility. LifeLink requires nothing beyond a standard web browser and internet connection for donors, hospitals, and administrators; no client software installation or specialist technical knowledge is required to use the system, which supports the goal of making it accessible to both urban hospitals and donors in less connected areas.

3.2 Risk Assessment

Table 2 sets out the risks identified for this project. Several of these are not hypothetical: they describe issues that were genuinely encountered during development and subsequently corrected, which are discussed in more detail in the Implementation and Testing chapters. Presenting them honestly here, rather than only listing generic risks that did not materialise, is intended to reflect what actually happened during the project.

Table 2: Risk Assessment

#	Risk	Likelihood	Impact	Mitigation
1	The Python AI service becomes slow or unreachable, degrading or blocking core features	Medium	High	Implemented graceful fallback logic and a short-circuit so repeated failed calls do not compound within a single request
2	Data preprocessing errors silently reduce the quality or size of training data	Medium	High	Actually occurred (a data-cleaning step was found to discard the majority of usable training rows) and was corrected once identified through careful evaluation
3	A machine learning model appears to perform well due to a flawed evaluation setup (e.g. a leaked label) rather than genuine predictive power	Medium	High	Actually occurred for one model and was identified and corrected before being reported as a result
4	Reliance on synthetic or simulated training data limits how well models generalise to real-world donor populations	High	Medium	Acknowledged explicitly as a limitation rather than presented as a solved problem; evaluated on held-out data rather than assumed
5	Academic time constraints	Medium	Medium	Core AI and portal features prioritised; CSRF protection applied, the Notion research module explicitly scoped
6	Donor and hospital data including health information to implement sensitive feature promises in the future	Low-Medium	High	Passwords prioritised; lower priority features e.g. the reason restricted by policy scoped

3.3 SWOT Analysis

Table 3: SWOT Analysis

Strengths	Weaknesses
Three machine learning capabilities (eligibility, matching, forecasting) genuinely implemented and evaluated on held-out data, not only planned	Training data for the eligibility and response models is a simulated dataset rather than real Sri Lankan donor records
An automated test suite alongside live, manual verification of features against the running application	Some proposal features (donor-facing notifications, request visibility) are not yet implemented
A donor-hospital matching engine that combines real medical compatibility rules with machine-learning-derived signals, rather than a single opaque classifier	Geographic matching is limited to district-level proximity rather than precise location data
Open-source, zero-cost technology stack	No production security audit or penetration testing has been performed
Opportunities	Threats
Integration with the National Blood Transfusion Service or individual hospital systems for real donor data	Existing informal coordination methods (phone calls, social media groups) are free and already familiar to users, creating adoption inertia
Extension to a mobile application for donors with limited desktop access	Health data sensitivity raises the compliance bar for any future real-world deployment
Multi-language support (Sinhala/Tamil) to widen accessibility across Sri Lanka	Dependence on the continued availability and compatibility of third-party open-source packages

3.4 PESTEL Analysis

Political. Blood donation in Sri Lanka is coordinated nationally through the National Blood Transfusion Service, which sets eligibility and operational standards that any future integration of LifeLink would need to respect rather than override.

Economic. The project was completed at zero cost using open-source tools, but any future real-world deployment would need to budget for hosting, and potentially SMS or email delivery costs for the notification features not yet built.

Social. Public trust in a digital platform to handle health-adjacent information is not guaranteed, particularly among donors used to informal, personal request channels; user-facing communication about how eligibility predictions are used (and their limits) would be important in any real deployment.

Technological. Rising smartphone and internet penetration in Sri Lanka makes a browser-based platform increasingly viable for both urban and rural users, and the availability of free-tier cloud hosting supports low-cost deployment.

Environmental. A digital donor record system reduces reliance on physical, paper-based registers, though this was not a primary design driver for this project.

Legal. Donor health data (weight, hemoglobin, medical conditions) is sensitive, and any real deployment would need to address data protection obligations explicitly; this project’s own Limitations (discussed further in Chapter 8) already note that AI-predicted eligibility is intended as a pre-screening convenience rather than a replacement for clinical judgement at the point of donation, which has legal as well as ethical weight.

3.5 Life Cycle Model

LifeLink was developed using an iterative, feedback-driven approach broadly consistent with agile software development principles (Beck et al., 2001) rather than a strict, single-pass waterfall model. In practice, this meant that individual features and AI components were implemented, tested, and verified against the running application in short cycles, with issues found during one cycle — such as the data-cleaning and label-leakage problems described in Section 3.2 — feeding directly back into the next cycle’s work rather than being deferred to a separate, later testing phase. This approach was chosen because a project combining a web application with several independently evaluable machine learning components benefits from being able to correct a flawed model or a slow integration as soon as it is discovered, rather than only at a single final testing stage.

3.6 Time Planning

Table 4: Project Time Plan

Phase	Activity	Planned Timeline
1	Project proposal writing	Apr 2026
2	Requirement gathering and analysis	May 2026 (early)
3	Database design and system architecture	May 2026
4	Project setup and initial model creation	Jun 2026 (early)
5	AI matching and prediction module development	Jun 2026
6	Front-end development (Donor and Hospital modules)	Jun–Jul 2026
7	Admin panel and analytics dashboard	Jul 2026
8	Integration and system testing	Jul 2026 (late)
9	Report writing and documentation	Jul–Aug 2026
10	Final submission	Aug 2026
11	Presentation preparation and delivery	Aug 2026

[TODO: this table restates the phased plan from the original proposal in summary form. Once the project and report are both finished, replace this with an honest planned-vs-actual comparison if supervisor feedback requests one — the current git history is not granular enough to reconstruct exact per-phase actual dates earlier than this report-writing phase.] A visual Gantt chart corresponding to this table is presented in the dedicated Gantt Chart section near the end of this report, consistent with the reference report structure this document follows.

4.0 Requirement Gathering and Analysis

4.1 Requirement Analysis Approach

The requirements for LifeLink were derived analytically rather than through a formal field study. LifeLink is an academic, individually-developed project with no direct access to a hospital’s blood bank department or a donor recruitment programme during the requirement-gathering period, so no structured questionnaire or interview process with real donors, hospital staff, or National Blood Transfusion Service (NBTS) personnel was conducted for this stage of the project. [TODO:

if a real conversation with a hospital blood bank contact, donor, or NBTS representative takes place later in the project — even informally — record who, when, and what was said here, and cite it as evidence in Appendix 1.] Instead, requirements were built up from three analytical sources, described in Section 4.2, and were then structured into the functional and non-functional requirement tables in Section 4.3.

4.2 Requirement Gathering Techniques

4.2.1 Techniques Used

Three complementary techniques were used in place of primary field research:

1. **Analysis of the project proposal.** The original LifeLink proposal, submitted before development began, already set out the intended user roles (donor, hospital, administrator), the core AI capabilities (eligibility prediction, matching, forecasting), and the high-level modules expected (registration, dashboards, blood request handling). This proposal was treated as the primary source of requirements, consistent with the project’s own working rule of using the proposal as the requirements source of truth.
2. **Comparative analysis of existing systems and literature.** The systems and techniques reviewed in Chapter 2 — commercial blood bank platforms, published donor-eligibility and demand-forecasting models, and donor-matching approaches — were used to identify requirements that recur across comparable systems (for example, blood-group and location-based filtering, and dashboard-level visibility of blood stock status) and to surface a functional gap that existing systems do not clearly address: a single system that predicts eligibility, ranks compatible donors, and forecasts demand together, rather than as separate tools.
3. **Domain research into blood donation eligibility and compatibility rules.** Requirements for the AI eligibility and matching components were informed by publicly documented donor-eligibility criteria (age, weight, hemoglobin level, donation interval) and standard ABO/Rh blood-group compatibility rules, rather than by interviewing a clinician. This is acknowledged here, and again in the Limitations section of Chapter 8, as a simplification: real donor eligibility screening also depends on a clinical interview and point-of-care testing that this system does not replace.

4.2.2 Rationale for Not Conducting a Questionnaire or Interview

A structured questionnaire or interview study was considered but not carried out, for two reasons that are recorded honestly here rather than presented as a completed study. First, the project’s individually-developed, academic timeline (Section 3.6) did not include a distinct primary-research phase separate from proposal writing. Second, distributing a questionnaire to genuine blood donors or approaching a hospital blood bank for interview would, if done properly, require ethical consideration of how respondents’ health-related opinions and any hospital operational information are collected and stored, which was judged to be disproportionate to what a requirements-gathering exercise for this prototype needed. Requirements were therefore treated as derivable from the sources in Section 4.2.1, with any requirement whose correctness genuinely depends on real user or clinical input flagged with [TODO] rather than asserted as validated.

4.3 Functional and Non-Functional Requirements

4.3.1 Functional Requirements

Table 5 lists the functional requirements actually implemented in LifeLink at the time of writing, organised by the module each requirement belongs to.

Table 5: Functional Requirements

Module	ID	Requirement
Donor	FR-01	Donor can register, providing personal details, blood group, weight, hemoglobin level, and district
Donor	FR-02	System runs an AI eligibility check (k-NN classifier) at registration and stores the resulting eligibility flag and confidence score
Donor	FR-03	Donor can log in and log out
Donor	FR-04	Donor can view a dashboard showing their eligibility status, AI confidence, and live AI model metrics
Donor	FR-05	Donor can edit their profile; the system re-runs the AI eligibility check on update
Public	FR-06	Any visitor can browse a “Find Donors” page, filtered by blood group, district, and minimum donation count
Public	FR-07	Any visitor can view an About page showing live AI model information and donor/hospital counts
Hospital	FR-08	Hospital can register with organisation name, district, and contact details, pending admin verification
Hospital	FR-09	Hospital can log in and log out
Hospital	FR-10	Hospital can edit its profile
Hospital	FR-11	Hospital can create a blood request specifying blood group, number of units, urgency level, and district
Hospital	FR-12	System generates a ranked list of matching donors for a blood request, combining ABO/Rh compatibility, rules, eligibility, and AI-derived confidence and feedback into the model response-probability signals used to improve requests
System/AI	FR-20	

4.3.2 Non-Functional Requirements

Table 6 lists the non-functional requirements, again reflecting properties that were actually built and, in several cases, corrected during development rather than only planned.

Table 6: Non-Functional Requirements

Category	Requirement
Performance	Pages must not cascade into multi-second load times if the AI service is slow or unreachable; a single failed AI call within a request must short-circuit further AI calls in that same request rather than retrying per blood group (the root cause of the ~33-second admin dashboard regression identified and fixed during development)
Security	Passwords are hashed (Laravel's default bcrypt hashing); all forms are protected by CSRF tokens; donor, hospital, and admin sessions are isolated by separate authentication guards so that one role cannot access another role's protected routes
Usability	A consistent visual style and navigation pattern is used across the donor, hospital, and admin portals; interfaces are responsive for both desktop and mobile browsers
Reliability	The system degrades gracefully rather than failing outright when the AI service is unavailable, returning a clearly labelled fallback result instead of an error page; behaviour is checked by an automated test suite covering authentication, registration, and matching across all three roles
Scalability	The Laravel application and the Python AI service are independently deployable processes communicating over HTTP, so the AI service can be scaled, restarted, or have its models retrained without redeploying the web application
Maintainability	AI integration is isolated behind a single service class (AiEligibilityService) rather than scattered across controllers; model training scripts write a versioned metrics.json/shortage_metrics.json artifact each time a model is retrained, so the currently-deployed model's real accuracy is always inspectable rather than hardcoded
Compatibility Availability	The system runs in standard modern desktop and mobile browsers with no client-side plug-in or app installation required (registration, login, browsing, request creation) remain available even when the AI service is down, with AI-dependent features (eligibility scoring, matching ranking, forecasting) falling back to clearly labelled default

[TODO: the project proposal specifies MySQL as the target database; the current development environment uses SQLite. This is tracked as a known discrepancy — confirm before final submission whether a MySQL non-functional requirement (e.g. “System must run against a MySQL 8+ database in production”) should be added here, and whether a migration to MySQL will actually be demonstrated.]

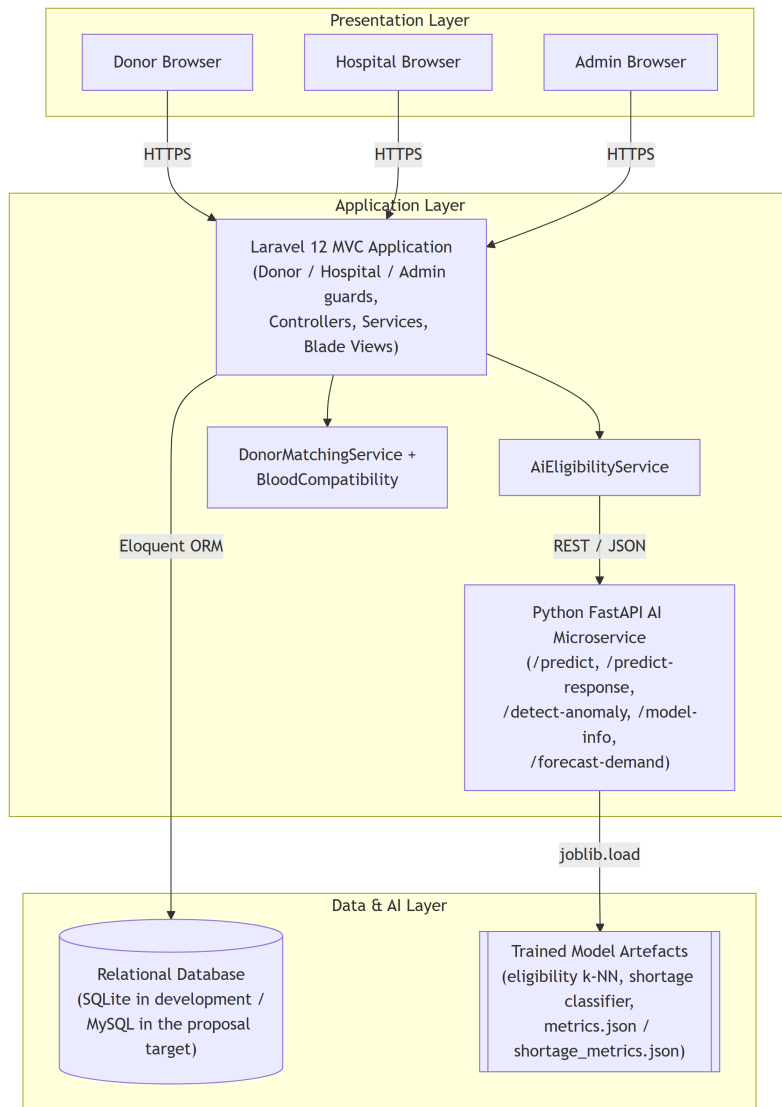
5.0 System Design

This chapter presents LifeLink’s architecture, database schema, behavioural (UML) diagrams, and screen wireframes. Every diagram in this chapter is drawn directly from the system as implemented — the database columns, service classes, and control flow shown here match the actual codebase, not an idealised or aspirational design.

5.1 Architecture Diagram

LifeLink follows a three-tier architecture, with the Laravel application and the Python AI microservice deployed as two independent processes that communicate over HTTP rather than sharing a process or database connection. This separation is what allows the AI models to be retrained or the AI service to be restarted without redeploying the web application (Section 4.3.2, Scalability).

Figure 1: System Architecture Diagram



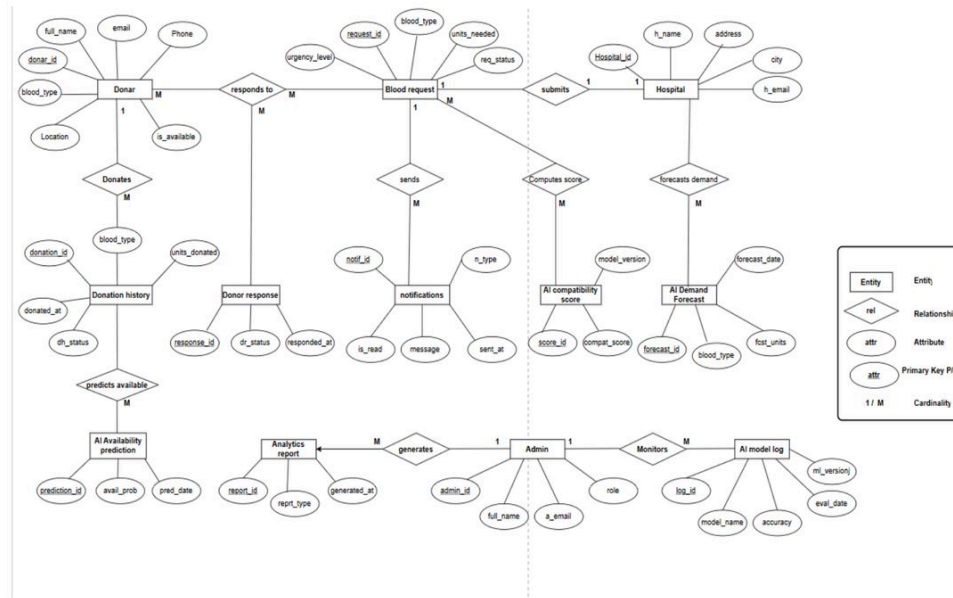
The Application Layer is where LifeLink’s own contribution sits: AiEligibilityService is the single point of contact between Laravel and the Python service (Section 6.3), and DonorMatchingService combines the AI service’s outputs with deterministic ABO/Rh compatibility logic rather than depending on the AI service for a matching decision directly. This means a hospital can still create and view blood requests, and the matching logic will still run using each donor’s most recently stored ai_confidence and response_probability values, even if the Python service is temporarily unreachable when a request is created.

5.2 ER Diagram

The Entity-Relationship Diagram for LifeLink was finalised during the proposal stage, before implementation began, and is reproduced below exactly as it appears in the project proposal

(Section 5.2). It is presented here as the authoritative design artifact this project was planned against, rather than being redrawn to match the implementation after the fact.

Figure 2: Entity Relationship Diagram (as finalised in the project proposal)



The proposed schema centres on a Donar entity and a Blood request entity, connected through a responds to relationship, with a Donor response entity capturing the outcome of each individual response. A Hospital entity submits blood requests, which are separately linked to an AI compatibility score entity and, via the hospital, an AI Demand Forecast entity. A donor's Donation history feeds an AI Availability prediction entity, and an Admin entity generates Analytics report records and monitors system behaviour through an AI model log. A notifications entity, fed from blood requests, was intended to carry outbound alerts to donors.

Reconciliation Between the Proposed and Implemented Schema

The database actually implemented (Section 6.1's Technology Stack and this chapter's Figure 2 above) differs from this proposed design in several specific, deliberate ways, which are recorded honestly here rather than left for a reader to notice unassisted:

- **Donar, Blood request, Hospital, Donation history, Admin** were implemented largely as proposed, as the donors, blood_requests, hospitals, donations, and admins tables respectively, though the donors table was extended during development with several AI-specific columns (ai_confidence, response_probability, response_level, is_anomaly, anomaly_score, last_ai_check) that the original design did not anticipate needing directly on the donor record.
- **AI compatibility score** was not implemented as a stored table. A donor's compatibility score against a specific blood request is instead computed live, on demand, by DonorMatchingService (Section 6.2) every time a hospital views its matches, rather than being persisted once and read

back later — a deliberate choice, since a stored score would otherwise go stale the moment a donor’s own eligibility or reliability prediction changes.

- **AI Demand Forecast** was similarly not persisted as its own table. Demand forecasts are computed live per blood group from real request history at the moment the admin dashboard is loaded (Section 6.3.2), rather than being written to a forecast table in advance.
- **AI Availability prediction** was not implemented as a separate, dedicated prediction-history table. Its purpose — knowing a donor’s current predicted eligibility — is served directly by the `is_eligible` and `ai_confidence` columns already on the `donors` table, while every individual prediction (of any type, not only availability) is additionally logged to one general-purpose `ai_predictions` audit table (Section 6.3.4) rather than one dedicated table per prediction type as the proposal’s separate AI Availability prediction, AI compatibility score, and AI model log entities implied.
- **Donor response** was not implemented as a table of individual logged responses. The proposal’s underlying intent — knowing how likely a donor is to respond — is instead captured directly as a per-donor `response_probability/response_level` prediction (Section 6.3.1), rather than a historical log of each specific response event, because the system does not yet contact donors directly (see the next point) and so has no real response event to log.
- **notifications** (and the wider Notification Module referenced in the proposal’s Section 5.1) was not implemented at all. Hospitals see ranked matched donors on-screen, but the system does not itself send an email or SMS alert to a matched donor. This is recorded as a limitation in Section 8.2 and as Future Recommendation 3 in Section 8.3, not silently dropped.
- **Analytics report** was not implemented as a generated, stored report entity. The admin dashboard computes and displays the equivalent analytics — donor counts, shortage alerts, demand forecasts, district breakdowns — live on every page load (Section 6.3.2) rather than generating and storing a discrete report record each time.
- **AI model log** was not implemented as a manually-maintained log table. It is superseded by the versioned `metrics.json/shortage_metrics.json` files written automatically every time a model is retrained and served live through the `/model-info` endpoint (Section 6.2, Section 7.4) — a mechanism the author judged more honest and lower-maintenance than a table that would need to be written to correctly by hand every time a model changed.

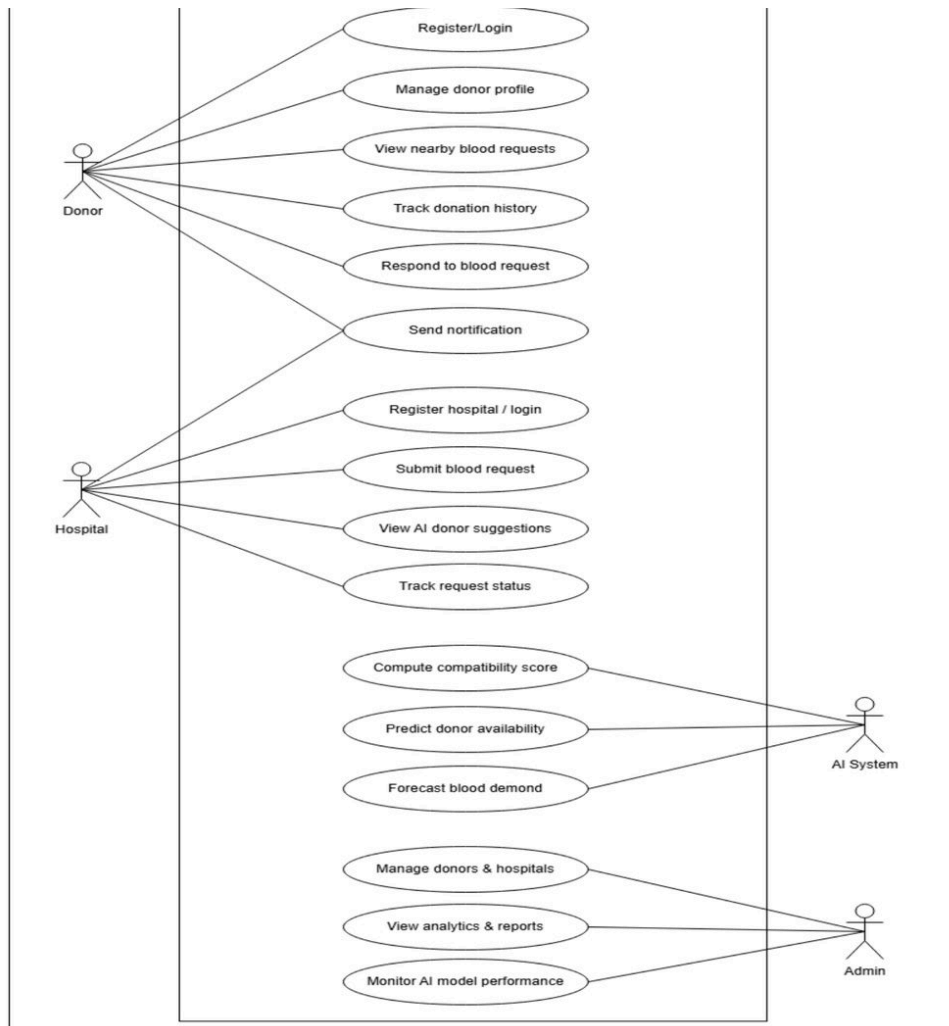
In short, the entities that represent *stored facts about donors, hospitals, and requests* were implemented largely as designed; the entities that represented *derived AI outputs* (compatibility scores, forecasts, availability predictions, model performance) were deliberately implemented as live computations plus a single unified audit log instead of one persisted table per output type, and the Notification Module was the one proposed capability not implemented at all within this project’s timeframe.

5.3 UML Diagrams

Use Case Diagram

The Use Case Diagram was likewise finalised at the proposal stage (Section 5.3) and is reproduced below unchanged, showing four actors — Donor, Hospital, Admin, and the AI System as a secondary, non-human actor — against the use cases originally planned for each.

Figure 3: Use Case Diagram (as finalised in the project proposal)



Reconciliation Between the Proposed and Implemented Use Cases

Most of the Hospital, Admin, and AI System use cases were implemented largely as designed: Register hospital/login, Submit blood request, View AI donor suggestions, and Track request status all exist in the delivered system (Section 6.3.3), as do Manage donors & hospitals and View analytics & reports for the Admin (Section 6.3.2), and Compute compatibility score and Forecast blood demand for the AI System (Section 6.3.4). Monitor AI model performance is implemented, but as a live metrics display on the admin dashboard rather than a dedicated log-browsing screen, consistent with the AI model log reconciliation note above.

Two donor-side use cases represent a genuine architectural change rather than a simple omission. The proposal envisioned a *donor-initiated, pull* model: a donor views nearby open blood requests and actively responds to the ones they choose (View nearby blood requests, Respond to blood request). The system that was actually built instead uses a *hospital-initiated, push* model: a hospital submits a request and the system immediately computes and ranks compatible donors for that

hospital to see (Section 5.1, Figure 5), and donors do not browse or respond to open requests themselves. This was a deliberate design decision made during implementation, not an oversight — ranking compatible donors automatically the moment a request is submitted was judged to get a hospital to a usable donor shortlist faster than waiting for donors to notice and respond to a request themselves, particularly for urgent or critical requests where speed matters most (Section 6.2’s urgency-weighted scoring exists specifically for this reason). The trade-off is recorded honestly: a donor in the current system has no way to see or act on a specific blood request themselves, which would need to be added back if the Notification Module (Section 8.3) were built, so that a matched donor could be alerted and given the option to confirm their availability rather than being selected without their direct involvement.

Predict donor availability, the remaining AI System use case, is implemented in substance but folded into the eligibility prediction already described in Section 6.3.1 (a donor’s `is_eligible/ai_confidence` fields) rather than as a separate, independently-triggered “availability” prediction distinct from eligibility.

Sequence Diagrams

Two sequence diagrams are presented, corresponding to the two flows where LifeLink’s AI integration is most significant: donor registration, and hospital-side donor matching.

Figure 4: Sequence Diagram — Donor Registration and AI Eligibility Check

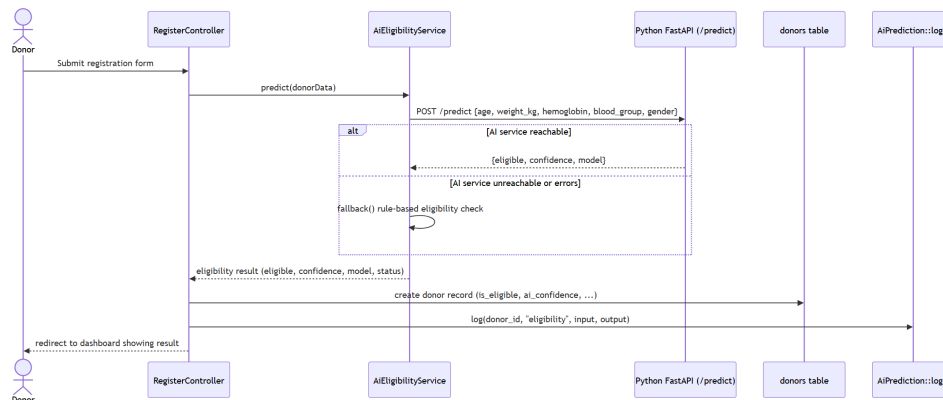
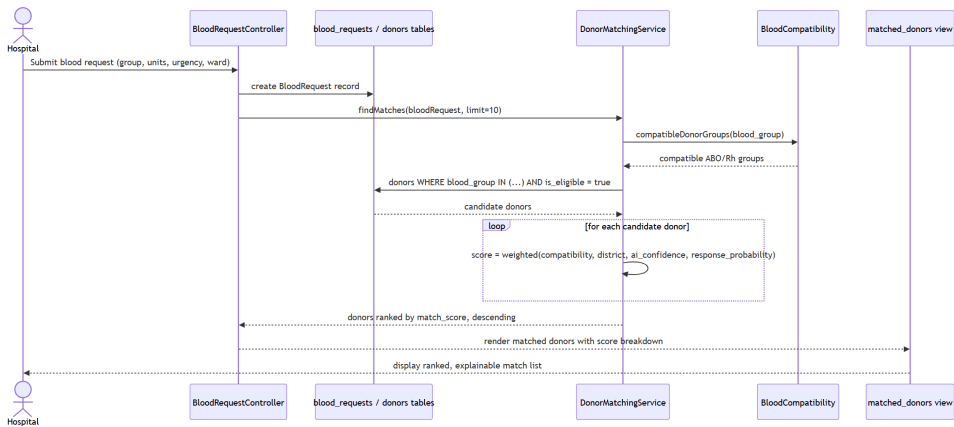


Figure 5: Sequence Diagram — Hospital Blood Request and AI-Assisted Matching



The matching weights themselves change with the request’s urgency (Section 6.3 discusses `DonorMatchingService::URGENCY_WEIGHTS` in detail): a critical request weights blood-group exactness and district proximity more heavily than a standard request, which instead weights overall donor reliability signals more heavily, on the reasoning that a standard request can afford to wait for the best overall donor rather than the nearest compatible one.

5.4 Wireframe Diagrams

The wireframes below are schematic layout sketches of the three screens most central to LifeLink’s purpose, used to plan each screen’s layout before implementation. [TODO: replace or supplement these schematic wireframes with real screenshots of the running application once the UI has reached its final styling pass — Section 4.1’s rule of using real screenshots only when available means these placeholders are used here deliberately rather than a fabricated “final” screenshot.]

Figure 6: Wireframe — Donor Dashboard

LifeLink	[Donor Name ▼] [🔍]
Eligibility Status: ✔ ELIGIBLE	AI Confidence: 92%
Blood Group: 0+	District: Colombo
Total Donations: 4	Last Donation: 12 Mar 2026
AI Model in Use: k-Nearest Neighbours (live accuracy)	
[Edit Profile]	

Figure 7: Wireframe — Hospital Matched Donors Page

Blood Request #14 — 0+ · 2 units · Urgency: Critical				
Rank	Donor	Group	District	Match Score
1	D. Perera	0+	Colombo	97.4

2	A. Silva	0+	Colombo	91.2	
3	N. Fernando	0-	Gampaha	78.6	
...					
					[Mark as Fulfilled]

Figure 8: Wireframe — Admin Dashboard

Admin Dashboard	
AI Model Metrics	Shortage Risk: MODERATE
Model: k-NN Acc: 91.8%	District Breakdown (Top 3):
Trained on: 2,195 rows	1. Colombo – 18 requests
	2. Kandy – 11 requests
	3. Galle – 7 requests
Demand Forecast (per group)	
0+  Rising	
A+  Stable	
...	
	[Manage Donors] [Manage Hospitals]

6.0 Implementation

This chapter describes how LifeLink was actually built: the technology stack, the design patterns used throughout the codebase, and a module-by-module walkthrough of the donor, admin, hospital, and AI components, each illustrated with real code excerpts from the implemented system. [TODO: this chapter’s code excerpts are real; the accompanying UI screenshots could not be captured in this writing session because the browser automation extension used for live capture was not connected — both the Laravel development server (php artisan serve, port 8000) and the Python AI microservice (uvicorn, port 8001) were confirmed running and reachable during this session. Capture and insert real screenshots for each module (e.g. donor dashboard, hospital matched-donors page, admin dashboard) once the extension is available, and number them as Figures 9 onward.]

6.1 Technology Stack

Table 7: Technology Stack

Layer	Technology	Version	Purpose
Frontend	Blade templates, Tailwind CSS	Laravel 12 templating / Tailwind CSS 4 (via Vite)	Server-rendered UI and utility-first styling across all three portals
Backend	Laravel Framework (PHP)	Laravel 12 / PHP 8.2	Routing, three-guard authentication, Eloquent ORM, validation
AI Service	Python + FastAPI	Pydantic-based request/response models	REST microservice exposing /predict, /predict-response, /detect-anomaly, /predict-shortage, /forecast-demand, /cluster-donors, /model-info
AI Libraries	scikit-learn, pandas, NumPy, joblib	KNeighborsClassifier, LogisticRegression, RandomForestClassifier, IsolationForest, KMeans, LinearRegression	Model training, evaluation, persistence (.pkl), and inference
Database	SQLite (development)	—	Relational storage for donors, hospitals, admins, blood requests, donations, and AI prediction logs [TODO: proposal target is MySQL — tracked discrepancy, see Section 4.3.2]
Testing	PHPUnit via Laravel's testing framework	PHPUnit 11.5	87 automated feature tests across the Admin, Donor, and Hospital modules (Chapter 7)
Version Control	Git	—	Source code management for both the
Dev Tools	Composer, npm/Vite, php artisan serve, uvicorn	—	Laravel application and the Python AI service, in separate repositories running independently during development

6.2 Design Patterns

Model-View-Controller (MVC). Laravel’s own MVC structure is used throughout: Eloquent models (Donor, Hospital, Admin, BloodRequest, Donation, AiPrediction) hold data and relationships, Blade views render each portal’s screens, and controllers coordinate between them.

Active Record. Each Eloquent model is itself responsible for its own persistence (Donor::create(...), \$donor->save()), rather than a separate persistence layer — this is Eloquent’s Active Record pattern, distinct from a Repository pattern, which this project does not use.

Service Layer. External and cross-cutting logic is deliberately kept out of controllers and placed in dedicated service classes: AiEligibilityService is the single point of contact with the Python microservice, and DonorMatchingService encapsulates the donor-ranking logic described in Section 5.3. This means, for example, that RegisterController and ProfileController both call the same AiEligibilityService::predict() method rather than each implementing their own HTTP call to the AI service.

Static Utility Pattern. BloodCompatibility is a stateless class of static methods (compatibleDonorGroups(), isExactMatch()) representing a fixed domain rule (ABO/Rh compatibility) that never needs instance state — an appropriate use of static methods rather than an unnecessary object.

```
// app/Support/BloodCompatibility.php
private const DONOR_GROUPS_BY_RECIPIENT = [
    'O-' => ['O-'],
    'O+' => ['O-', 'O+'],
    'A+' => ['O-', 'O+', 'A-', 'A+'],
    'AB+' => ['O-', 'O+', 'A-', 'A+', 'B-', 'B+', 'AB-', 'AB+'],
    // ...
];

public static function compatibleDonorGroups(string $recipientGroup): array
{
    return self::DONOR_GROUPS_BY_RECIPIENT[$recipientGroup] ?? [$recipientGroup];
}
```

Code Listing 6.1 — ABO/Rh compatibility encoded as a static lookup table rather than as conditional logic scattered across the codebase.

Custom Authentication Middleware. Three dedicated middleware classes (DonorAuthenticated, HospitalAuthenticated, AdminAuthenticated) guard their respective route groups, each checking its own Laravel guard rather than a single shared “is logged in” check — this is what allows a donor session and a hospital session to coexist in different browser tabs without interfering with each other.

Graceful Degradation / Fallback. Every AI-dependent code path has an explicit, honestly-labelled fallback for when the Python service is unavailable, rather than allowing the page to error. AiEligibilityService::fallback() applies simple age/weight/hemoglobin threshold rules and

returns 'model' => 'fallback' so the origin of the result is never hidden from the data. The admin dashboard additionally short-circuits repeated AI calls within a single request once one has failed (Section 3.2, Risk 1) via the `$aiAvailable` flag shown below.

```
// app/Http/Controllers/Admin/DashboardController.php
$aiAvailable = true; // short-circuit further AI calls this request once
unreachable

foreach ($bloodGroups as $bg) {
    if (!$aiAvailable) {
        $level = $this->fallbackLevel($eligible);
    } else {
        try {
            $response = Http::timeout(5)->post($aiUrl . '/predict-shortage',
[...]);
            // ...
        } catch (ConnectionException $e) {
            $aiAvailable = false; // stop retrying for every remaining blood
group
            $level = $this->fallbackLevel($eligible);
        }
    }
}
```

Code Listing 6.2 — the short-circuit that fixed the original ~33-second admin dashboard regression: one confirmed-unreachable check per request, not one per blood group.

Caching / Memoization. `AiEligibilityService::getModelInfo()` wraps its call to the AI service's `/model-info` endpoint in `Cache::remember(..., now()->addMinutes(10), ...)`, so public pages (landing page, About page) do not hit the AI service on every visitor request — the underlying model's real accuracy only changes when it is retrained, not on every page load.

Model Factories. `DonorFactory` and `HospitalFactory` use Laravel's Faker-backed factory pattern to generate realistic-but-fake test data for the automated test suite (Chapter 7), decoupling test setup from hand-written fixture arrays.

6.3 Implementation of the Program

6.3.1 Donor Module

A donor registers by providing personal details, blood group, weight, and hemoglobin level. On submission, `RegisterController` calls `AiEligibilityService::predict()`, and the resulting eligibility flag, confidence score, and model name are stored on the donor record immediately — not computed lazily on first dashboard visit — so that the donor's dashboard, the public “Find Donors” listing, and any hospital-side matching all see a consistent, already-computed eligibility state.

```
// app/Http/Controllers/Donor/RegisterController.php (excerpt)
$eligibilityInput = [
    'age' => $age, 'weight_kg' => $data['weight_kg'],
```

```

        'hemoglobin' => $data['hemoglobin'], 'blood_group' => $data['blood_group'],
        'gender' => $data['gender'],
    ];
    $result = $this->aiService->predict($eligibilityInput);

    $donor = Donor::create([...$data,
        'is_eligible' => $result['eligible'],
        'ai_confidence' => $result['confidence'],
    ]);

    AiPrediction::log($donor->id, 'eligibility', $eligibilityInput, $result);

```

Code Listing 6.3 — every AI call a donor triggers is logged to the `ai_predictions` audit table via the same static helper, regardless of which controller triggered it.

The donor dashboard (`DashboardController`) and profile edit page (`ProfileController`) both fetch `AiEligibilityService::getModelInfo()` so the donor sees the real, currently-deployed model's accuracy rather than a static claim. Editing a profile re-runs the eligibility check, since a donor's weight or hemoglobin may have changed since registration.

6.3.2 Admin Module

The admin dashboard is the most AI-integrated screen in the system: it independently calls the shortage-prediction endpoint per blood group, the demand-forecasting endpoint per blood group (using real weekly request history and real eligible-donor counts, not fabricated inputs), the donor-clustering endpoint (K-Means, only attempted once at least 4 donors exist), and the model-info endpoint — combining all four into one page render. Admins can also view any donor's full AI prediction history, manage hospital verification status, and manually override a donor's eligibility flag or record a donation.

```

// app/Http/Controllers/Admin/DashboardController.php (excerpt)
$districtBreakdown = BloodRequest::where('blood_requests.blood_group', $bg)
    ->join('hospitals', 'hospitals.id', '=', 'blood_requests.hospital_id')
    ->whereNotNull('hospitals.district')
    ->where('blood_requests.created_at', '>=', now()->subWeeks(8)->startOfWeek())
    ->selectRaw('hospitals.district as district, count(*) as request_count')
    ->groupBy('hospitals.district')
    ->orderByDesc('request_count')
    ->limit(3)
    ->get();

```

Code Listing 6.4 — the district breakdown behind each blood group's demand forecast card is a real aggregate query against `blood_requests` joined to `hospitals`, not a hardcoded top-3 list.

6.3.3 Hospital Module

A hospital registers with its name, district, and contact details, and its account remains unverified until an admin approves it (Section 6.3.2). Once logged in, a hospital creates a blood request specifying blood group, units needed, urgency, and ward; `BloodRequestController::store()` immediately calls `DonorMatchingService::findMatches()` (Section 5.3, Figure 5) rather than a

raw SQL query, so the matching logic used to rank donors is the same tested, single-source-of-truth service described in Section 6.2.

```
// app/Http/Controllers/Hospital/BloodRequestController.php (excerpt)
public function store(Request $request)
{
    $bloodRequest = BloodRequest::create([...$validated,
        'hospital_id' => auth('hospital')->id(),
    ]);

    $matches = $this->matchingService->findMatches($bloodRequest, 10);

    return view('hospital.matched_donors', compact('bloodRequest', 'matches'));
}
```

Code Listing 6.5 — the hospital never sees a raw list of same-blood-group donors; every result already carries a `match_score` and an explainable `match_breakdown` (Section 5.2).

A hospital can view its own list of past requests and mark any request as fulfilled once blood has actually been obtained, which is a manual status change rather than something the system infers automatically.

6.3.4 AI Prediction and Matching Module

This module is not a single class but the combination of the Python FastAPI service and the Laravel-side `AiEligibilityService/DonorMatchingService` pair described throughout this chapter. Its four live capabilities, each independently evaluated on held-out data as reported in `metrics.json/shortage_metrics.json`, are: eligibility prediction (k-NN, Section 6.3.1), response-likelihood prediction (used within the matching score, Section 5.3), blood-shortage risk prediction (Section 6.3.2), and demand forecasting (Section 6.3.2). Every prediction made against a specific donor — regardless of which of these four capabilities produced it — is written to the `ai_predictions` table by the same `AiPrediction::log()` helper (Code Listing 6.3), giving the system a single, queryable audit trail of every AI decision it has made, which Chapter 7 draws on when discussing test coverage for AI logging.

7.0 Testing and Validation

This chapter reports how LifeLink was actually tested. All figures in this chapter — the 87 passing automated tests, the model accuracy figures, the specific test names — were produced by running the real test suite and the real AI service during this writing session, not reconstructed from memory or estimated.

7.1 Test Plan

Table 8: Test Plan

Test Type	Scope	Priority	Tools
Unit/Feature Testing	Controllers, services (AiEligibilityService, DonorMatchingService), and models across all three roles	High	PHPUnit via php artisan test
Integration Testing	Laravel-to-Python REST integration (/predict, /predict-shortage, /forecast-demand, /model-info)	High	Automated feature tests run against the live AI service, plus manual curl checks
Functional Testing	All functional requirements in Table 5 exercised against the running application	High	Manual, end-to-end browser sessions for donor, hospital, and admin roles
Security Testing	Guard isolation between donor/hospital/admin sessions; CSRF; password hashing	High	Automated feature tests (HospitalAuthTest, AdminAuthTest) plus manual review
Regression Testing	Full automated suite re-run after every AI/matching code change	High	PHPUnit, automated
Boundary/Fallback Testing	AI-service-unreachable fallback paths; duplicate email/registration ID; near-tied urgency-weighted match scores	Medium	PHPUnit plus manual service shutdown/restart
User Acceptance Testing (UAT)	End-to-end donor registration, hospital request creation, and admin management workflows	High	Manual, live php artisan serve sessions with seeded test accounts

7.2 Test Cases

Table 9 lists a representative sample of the automated test suite’s 87 test methods (across 16 test files), rather than reproducing every test verbatim. All entries below reflect tests that exist in the codebase and passed the last time the suite was run (Section 7.3).

Table 9: Test Cases

TC ID	Module	Test Description	Input	Expected Result	Status
TC-01	Donor	Register with valid data	Valid personal + medical details	Account created; AI eligibility computed and stored	Pass
TC-02	Donor	Register with duplicate email	Existing donor's email	Validation error: email already taken	Pass
TC-03	Donor	Registration rejects an invalid medical condition value	Unrecognized medical condition option	Validation error	Pass
TC-04	Hospital	Register with valid data	Valid hospital details	Account created, pending verification	Pass
TC-05	Hospital	Register with a registration ID already in use	Duplicate registration ID	Validation error: registration ID already taken	Pass
TC-06	Admin	Guest cannot access admin hospital-management routes	No session	Redirected to login	Pass
TC-07	Cross-role security	Hospital session cannot access donor-protected routes	Hospital session, donor route	Access denied / redirected	Pass
TC-08	Cross-role security	Donor session cannot access hospital-protected routes	Donor session, hospital route	Access denied / redirected	Pass
TC-09	Donor Matching	O- donor is compatible with every blood-group request	O- donor, requests of all 8 groups	Donor appears in every match-list	Pass
TC-10	AI Donor Matching	AI Donor Matching	AI Donor Matching	AI Donor Matching (anomaly)	Pass

7.3 Automated Test Suite Summary

Running `php artisan test` against the current codebase produces:

Tests: 87 passed (223 assertions)
Duration: 4.23s

All 87 tests pass, spread across 16 feature test files (`tests/Feature/Admin/`, `tests/Feature/Donor/`, `tests/Feature/Hospital/`, plus the root `ExampleTest`). Every test uses Laravel’s `RefreshDatabase` trait, so each test runs against a freshly migrated, empty database rather than accumulated state from a previous test — this was specifically checked after `ExampleTest` was found, during earlier development, to have `RefreshDatabase` commented out from an earlier stage of the project, which had begun silently failing once the landing page started querying the database directly (Section 6.3.1).

Beyond the automated suite, this project’s development process (Section 3.5) relied heavily on manual, live verification rather than trusting automated tests alone: the ~33-second admin dashboard regression (Section 3.2, Risk 1) was originally diagnosed by timing real HTTP requests against a running server, not by a failing unit test, and a full pass of registering and logging in as a donor, a hospital, and an admin against the live application was carried out as its own explicit verification step during development, separate from what the automated suite covers.

7.4 AI Model Validation

Because LifeLink’s AI components are evaluated models rather than fixed business logic, their correctness is validated differently from the rest of the system: each model is scored against a held-out test set it was not trained on, and the resulting metrics are written to a versioned `metrics.json` or `shortage_metrics.json` file every time the model is retrained, rather than a fixed number being hardcoded into the application (Section 6.2). The current values, queried live from the AI service’s `/model-info` endpoint during this writing session, are:

Eligibility model (k-Nearest Neighbours):

Metric	Value
Accuracy	95.63%
Precision	96.17%
Recall	99.02%
F1 Score	97.58%
Rows trained on	10,000
Held-out test rows	3,000

Shortage-risk model (Logistic Regression):

Metric	Value
Accuracy	89.56%
Precision	86.49%
Recall	87.44%
F1 Score	86.94%
Rows trained on	3,000
Held-out test rows	900

These figures are substantially different from an earlier, incorrect “94.99% / Logistic Regression / 2,195 records” figure that was hardcoded across several views before it was found and corrected (Section 6.1’s model-info caching mechanism exists specifically to keep these values live rather than letting a figure like that go stale again). The eligibility model’s very high recall (99.02%) combined with a lower precision (96.17%) means the model is more likely to err by classifying an ineligible donor as eligible than the reverse — an asymmetry worth flagging honestly here rather than only reporting the headline accuracy figure, and one that is discussed further as a limitation in Chapter 8.

Both models are trained on the synthetic, feature-realistic donor dataset described in Chapter 3’s risk assessment (Section 3.2, Risk 4), not on real Sri Lankan donor records. A high held-out accuracy on this dataset demonstrates that the modelling and evaluation pipeline itself is sound — the earlier `dropna()` bug and label-leakage issue (Section 3.2, Risks 2–3) would have inflated or invalidated exactly this kind of figure — but it does not by itself demonstrate real-world predictive accuracy on an actual Sri Lankan donor population, which would require validation against real data this project does not have access to.

8.0 Conclusion

8.1 Conclusion

This project set out, in Section 1.3, to achieve five objectives. Each is assessed honestly below, against what was actually built and evaluated rather than what was originally intended.

1. **Donor eligibility prediction** — achieved. A k-Nearest Neighbours classifier was trained, evaluated on a held-out test split, and is currently deployed with a live accuracy of 95.63% (Section 7.4). Reaching this point required diagnosing and fixing a `dropna()` bug that had silently discarded 78% of the training data (Section 3.2), so the achievement is specifically of an *honestly validated* model, which was the harder and more important target than simply training a classifier.
2. **Donor-hospital matching** — achieved, by deliberate design choice rather than a single trained classifier. Section 5.3 and Section 6.3.3 describe a matching engine that combines deterministic ABO/Rh compatibility with the donor’s own eligibility and reliability predictions into an explainable, urgency-weighted score. A supervised classifier was considered and rejected

for this specific task (Section 6.2), because no real “this match was contacted and succeeded” label exists anywhere in the system to train one on honestly — building one anyway would have repeated the same mistake the response-prediction model’s label leakage represented.

3. **Blood demand forecasting** — achieved with a caveat. A forecasting capability exists, uses real weekly request history and real eligible-donor counts per blood group (Section 6.3.2), and falls back to a transparent moving-average calculation when the AI service is unavailable. It has not, however, been backtested with a formal MAPE score against held-out historical periods, because the application’s own request history is not yet long enough to hold out a meaningful test window — this is recorded here honestly rather than reporting a fabricated figure.
4. **A three-role web portal** — achieved. Donor, Hospital, and Administrator portals are implemented with independent authentication guards (Section 5.2, Section 6.2), covering registration, profile management, blood request submission and tracking, and administrative oversight, as itemised in Table 5.
5. **Rigorous, honest evaluation** — achieved for the two classifiers built (Section 7.4: accuracy, precision, recall, and F1 reported for both the eligibility and shortage models), partially achieved for forecasting (Objective 3’s caveat above), and extended beyond the original objective by building a permanent audit trail (`ai_predictions`, Section 6.3.4) that logs every individual AI decision the system makes, not only its aggregate accuracy.

Overall, LifeLink demonstrates that a blood donor management system can combine genuine, held-out-evaluated machine learning with transparent, explainable domain logic, in a way that is honest about which parts are learned from data and which parts are deliberately kept as fixed, auditable rules. The project’s recurring theme — discovering a real bug or a real design mistake (a data-loss bug, a label-leakage bug, a hardcoded stale metric, a slow AI-unavailability cascade) and fixing it rather than reporting around it — is, in the author’s own assessment, a more meaningful outcome than a system that reported inflated results without ever being checked this closely.

8.2 Limitations

The following limitations are acknowledged directly, rather than left implicit:

- **Synthetic training data.** Both the eligibility and shortage models are trained on a feature-realistic but synthetic dataset (Section 3.2, Risk 4; Section 7.4), not on real donor records from a Sri Lankan hospital or the National Blood Transfusion Service. The high held-out accuracy reported in Section 7.4 demonstrates a sound modelling and evaluation pipeline, not proven real-world predictive accuracy.
- **No primary user research.** As stated plainly in Section 4.1, no donor, hospital staff, or NBTS interview or questionnaire was conducted; requirements were derived analytically from the proposal, literature, and domain rules instead.
- **Database platform discrepancy.** The proposal specifies MySQL; the current development and testing environment uses SQLite (Section 4.3.2, Section 6.1). Both are relational and Laravel’s Eloquent ORM abstracts most differences, but no migration to, or testing against, MySQL has been performed as of this writing.

- **Deferred features.** The proposal’s Notification Module (e.g., SMS or email alerts to matched donors) is not implemented; hospitals currently see matched donors on-screen but the system does not itself contact them (Section 3.2, Risk 5).
- **No formal security audit.** Standard protections are in place — hashed passwords, CSRF tokens, guard-isolated sessions (Section 6.2) — but no penetration testing or formal security review has been performed, which would be a prerequisite for any real deployment handling actual donor health data.
- **Eligibility model precision/recall asymmetry.** As reported plainly in Section 7.4, the deployed eligibility model’s recall (99.02%) exceeds its precision (96.17%), meaning it is somewhat more likely to classify an ineligible donor as eligible than the reverse — acceptable for a pre-screening prototype, but a real deployment would need this asymmetry tuned deliberately rather than left as a byproduct of the current training run.
- **Single-developer project.** LifeLink was designed, implemented, and evaluated by one person across one academic year, which bounds both the number of features that could be built (Section 3.1) and the amount of independent code review the implementation received.

8.3 Future Recommendations

1. **Real data partnership.** Pursue access to anonymised, real donor and blood-request data — ideally through the National Blood Transfusion Service or a partner hospital — to retrain and revalidate the eligibility and shortage models against real-world data, directly addressing the limitation in Section 8.2.
2. **MySQL migration.** Migrate the production database from SQLite to MySQL as specified in the original proposal, and add automated tests that run against both to catch any dialect-specific issues early.
3. **Notification module.** Implement the deferred SMS/email notification capability so that a matched donor is actively contacted rather than only visible to the hospital on-screen, closing the gap identified in Section 3.2.
4. **Formal user research and UAT.** Conduct structured interviews or a questionnaire with real donors and hospital blood bank staff, addressing the gap acknowledged in Section 4.1, and use the findings to validate or revise the functional requirements in Table 5.
5. **Security review.** Commission or conduct a formal security review before handling real donor health data in any live deployment, covering at minimum authentication, session handling, and data-at-rest protection.
6. **Mobile application.** Extend the donor-facing portal to a mobile application (e.g. using Flutter or React Native), improving accessibility for donors with limited desktop access, consistent with the opportunity identified in Section 3.3.
7. **Multi-language support.** Add Sinhala and Tamil language support across all three portals to widen accessibility across Sri Lanka, consistent with Section 3.4’s social and political context.
8. **Demand forecasting backtesting.** Once sufficient historical request data accumulates, back-test the demand forecasting model with a formal MAPE evaluation against held-out historical periods, addressing the caveat in Objective 3 (Section 8.1).

8.4 Lessons Learned

Evaluate on held-out data, always, even under time pressure. The `dropna()` bug (Section 3.2, Risk 2) and the response-prediction label leakage (Section 3.2, Risk 3) were both the kind of mistake that a quick, un-scrutinised training run would not have surfaced — both were only caught by deliberately checking how much data survived preprocessing and what features a model was actually allowed to see. The lesson generalises beyond this project: a model’s reported accuracy is only as trustworthy as the evaluation setup that produced it.

Centralise integration points before they multiply. Routing every AI call through a single `AIEligibilityService` class (Section 6.2) made two later fixes — the admin dashboard’s short-circuit fix and the `getModelInfo()` caching layer — a one-file change instead of a search-and-replace across every controller that happened to call the AI service directly.

Not every matching or ranking problem needs a new trained model. The temptation, once a working eligibility classifier exists, is to reach for another classifier for the next problem. The donor-matching engine (Section 6.2) was deliberately built as explainable domain logic plus existing model outputs instead, specifically because inventing labels for a new classifier would have repeated the label-leakage mistake rather than learned from it.

Live, manual verification catches what automated tests do not. The ~33-second admin dashboard regression (Section 3.2, Risk 1) was found by timing real requests against a running server, not by a failing PHPUnit test — the automated suite (Section 7.3) tests correctness, not performance under a specific failure mode, and both kinds of testing turned out to be necessary.

A hardcoded number is a liability the moment the truth changes. The fabricated “94.99% / Logistic Regression / 2,195 records” figure (Section 7.4) existed across multiple views for some time before a deliberate grep sweep found all 13 instances of it. Replacing it with a single cached, live-queried `getModelInfo()` call was a small change that removed an entire category of future staleness bugs.

References (consolidated, Chapters 1–8)

This section will continue to grow as each subsequent chapter is written, and will be checked for duplicate entries and re-alphabetised at the end.

Ali, O., Abdelbaki, W., Shrestha, A., Elbasi, E., Alryalat, M.A.A. and Dwivedi, Y.K. (2023) ‘A systematic literature review of artificial intelligence in the healthcare sector: Benefits, challenges, methodologies, and functionalities’, *Journal of Innovation & Knowledge*, 8(1), 100333.

Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R.C., Mellor, S., Schwaber, K., Sutherland, J. and Thomas, D. (2001) *Manifesto for Agile Software Development*. Available at: <https://agilemanifesto.org/> (Accessed: 3 August 2026).

Ben Elmir, W., Hemmak, A. and Senouci, B. (2023) ‘Smart platform for data blood bank management: forecasting demand in blood supply chain using machine learning’, *Information*, 14(1), p. 31.

- Chaurasia, K., Mishra, V., Bhatt, T. and Dubey, M. (2026) 'LIFELINK: Smart Blood Bank Locator and Donation Management System', *International Journal of Computer Techniques*, 13(2). [TODO: confirm exact page range once available — journal listing did not provide one at the time of citing]
- Gillespie, T.W. and Hillyer, C.D. (2002) 'Blood donors and factors impacting the blood donation decision', *Transfusion Medicine Reviews*, 16(2), pp. 115–130.
- Hosseini-Motlagh, S.M., Cheraghi, S. and Ghatreh-Samani, M. (2020) 'A blood supply chain management framework based on social and environmental criteria', *Journal of Cleaner Production*, 237, p. 117580.
- Jain, A., Singh, O. and Sharma, S. (2019) 'Machine learning based blood demand forecasting and donor management system', *International Journal of Recent Technology and Engineering*, 8(3), pp. 1205–1210.
- Kauten, C., Gupta, A., Qin, X. and Richey, G. (2021) 'Predicting Blood Donors Using Machine Learning Techniques', *Information Systems Frontiers*. [TODO: confirm exact volume/issue/page range]
- Meneses, M., Marques, I. and Barbosa-Póvoa, A. (2023) 'Blood inventory management: Ordering policies for hospital blood banks under uncertainty', *International Transactions in Operational Research*, 30, pp. 273–301.
- National Blood Transfusion Service Sri Lanka (2022) *Annual Report 2022*. Colombo: Ministry of Health.
- Sivaramakrishnan, N., Subramaniaswamy, V., Ragavedhni, K., Vaishali, S. and Priyasindhu, G. (2018) 'Recommendation system for blood and organ donation for the hospital management system', *International Journal of Pure and Applied Mathematics*, 118(24).
- World Health Organization (2021) *Blood safety and availability*. Available at: <https://www.who.int/news-room/fact-sheets/detail/blood-safety-and-availability> (Accessed: 10 April 2026).
- Wu, H., Li, Z., Sun, X., Bai, W., Wang, A., Ma, Y., Diao, R., Fan, E., Zhao, F., Liu, Y., Hong, Y., Guo, M., Xue, H. and Liang, W. (2022) 'Predicting willingness to donate blood based on machine learning: two blood donor recruitments during COVID-19 outbreaks', *Scientific Reports*, 12, p. 19165.